# Data Structures (ETCCDS202)

## Unit–1 Assignment: Foundations & Algorithmic Analysis

## Course Details

| | |
|---|---|
| Name of the School: | School of Engineering & Technology |
| Program (Class/Semester/Batch): | B.Tech AIML (Sec-A, Sem-II, 2025-29) |
| Course Title: | Data Structures |
| Course Code: | ETCCDS202 |
| Unit Title: | Foundations & Algorithmic Analysis |
| Student Name: | Daksh Dua |
| Roll No.: | 2501730141 |
| Section: | A |
| Submission Date: | 4th March 2026 |

# 1. Stack ADT

An Abstract Data Type (ADT) defines a data structure based on the operations that can be performed on it, without explaining how those operations are implemented internally. It focuses more on “what it does” rather than “how it does it”.

In this assignment, I implemented a Stack ADT. A stack follows the **Last-In-First-Out (LIFO)** principle, which means the last element inserted is the first one to be removed. The main operations of a stack are:

- push(x) – insert an element
- pop() – remove the top element
- peek() – view the top element
- is_empty() – check whether the stack is empty
- size() – return the number of elements

All these operations run in **O(1)** time because they only operate at the top of the stack and do not require traversal of the entire structure.

# 2. Recursive Factorial

The factorial of a number n is defined as:

$n! = n \times (n-1) \times (n-2) \times \ldots \times 1$
with the base case:
$0! = 1$

The recursive solution works by reducing the problem into smaller subproblems:

- If n = 0 or 1 → return 1 (base case)
- Otherwise → return n × factorial(n−1)

## Time Complexity

Each recursive call reduces n by 1, and there are exactly n calls before reaching the base case.

So, the recurrence can be written as:

$T(n) = T(n-1) + O(1)$

Therefore:

- **Big-O = O(n)**
- **Big-Θ = Θ(n)**
- **Big-Ω = Ω(n)**

Since the number of operations grows linearly with n, the best, average, and worst cases are all linear.

## Space Complexity

Because recursion is used, each function call is stored in the call stack. The maximum depth of recursion is n.

So, space complexity is:

**O(n)**

# 3. Fibonacci – Naive vs Memoized

The Fibonacci sequence is defined as:

$F(n) = F(n-1) + F(n-2)$
with base cases:
$F(0) = 0$
$F(1) = 1$

## 3.1 Naive Recursive Fibonacci

The naive approach directly follows the mathematical definition. However, it recalculates the same Fibonacci values multiple times. For example, F(5) calculates F(3) and F(2) repeatedly.

This leads to overlapping subproblems.

### Time Complexity

The recurrence relation is:

$T(n) = T(n-1) + T(n-2) + O(1)$

This results in exponential growth.

- **Big-O = $O(2^n)$**
- **Big-Θ = $\Theta(2^n)$**
- **Big-Ω = $\Omega(2^n)$**

The number of recursive calls increases very rapidly as n increases, which makes this approach inefficient for large values of n.

### Space Complexity

The recursion depth is n, so:

**O(n)**

## 3.2 Memoized Fibonacci (Dynamic Programming)

In the memoized version, previously computed Fibonacci values are stored in a dictionary. If a value has already been calculated, it is returned immediately instead of being recomputed.

This removes overlapping subproblems.

### Time Complexity

Each Fibonacci number from 0 to n is computed only once.

$T(n) = T(n-1) + O(1)$

So:

- **Big-O = O(n)**
- **Big-Θ = Θ(n)**

- **Big-Ω = Ω(n)**

This is a huge improvement compared to the naive version.

### Space Complexity

We use:

- O(n) space for storing computed values
- O(n) recursion stack depth

So total space complexity is:

**O(n)**

# 4. Tower of Hanoi

The Tower of Hanoi problem involves moving n disks from one rod to another using an auxiliary rod, following certain rules:

1. Only one disk can be moved at a time.
2. A larger disk cannot be placed on a smaller disk.

The recursive idea is:

- Move n−1 disks to auxiliary rod
- Move the largest disk to destination
- Move n−1 disks from auxiliary to destination

## Recurrence Relation

$T(n) = 2T(n-1) + 1$

Solving this gives:

$T(n) = 2^n - 1$

## Time Complexity

Since the number of moves grows exponentially:

- **Big-O = $O(2^n)$**
- **Big-Θ = $\Theta(2^n)$**
- **Big-Ω = $\Omega(2^n)$**

## Space Complexity

The maximum recursion depth is n.

So:

**O(n)**

# 5. Recursive Binary Search

Binary search works only on sorted arrays. It repeatedly divides the array into two halves and searches only in the relevant half.

Steps:

1. Find the middle element.
2. If it matches the key → return index.
3. If key is smaller → search left half.
4. If key is larger → search right half.

## Recurrence Relation

$T(n) = T(n/2) + O(1)$

Since the problem size is reduced by half each time, the solution becomes:

$T(n) = O(\log n)$

## Time Complexity

- Best Case: O(1) (key found at middle)
- Average Case: O(log n)
- Worst Case: O(log n)

## Space Complexity

Since recursion is used, the depth of recursion is:

O(log n)

# 6. Algorithm Paradigm Reflection

## Binary Search – Divide & Conquer

Binary search follows the Divide & Conquer paradigm because it divides the problem into two smaller subproblems (halves), solves one of them recursively, and reduces the search space at each step.

## Fibonacci (Memoized) – Dynamic Programming

The memoized Fibonacci solution uses Dynamic Programming because it solves overlapping subproblems and stores previously computed results. This avoids repeated work and improves efficiency significantly.

## Tower of Hanoi – Recursion

Tower of Hanoi is a classic example of recursion. It has:

- A clear base case (n = 1)
- A recursive case that solves a smaller version of the same problem

The same logic repeats until the base case is reached.

# 7. Real-Life Examples of Algorithm Paradigms

**Divide & Conquer:** Searching for a name in a dictionary by opening it in the middle and deciding which half to search.

**Dynamic Programming:** Saving shortest paths in navigation systems so that the same distances are not recalculated repeatedly.

**Greedy Approach:** Giving change using the largest available currency note first.

# Conclusion

Through this assignment, I understood how recursion works internally using the call stack and how algorithm efficiency can vary significantly depending on implementation. The comparison between naive and memoized Fibonacci clearly demonstrates the importance of avoiding overlapping subproblems. Understanding time and space complexity helps in selecting the most efficient solution for practical applications.